

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Towards the Optimization of General Logistics Scenarios with Multi-Agent Simulations: Results of the AbstractSwarm Multi-Agent Logistics Competition with Application in Hospital Logistics

DAAN APELDOORN¹, ALINA JENN¹, ALEXANDER DOCKHORN², (Member, IEEE), JOCHEN WOLLSTÄDTER³, FABIAN LINKERHÄNGER, and TORSTEN PANHOLZER¹

¹Medical Informatics, Institute of Medical Biostatistics, Epidemiology and Informatics (IMBEI), University Medical Center of the Johannes Gutenberg University, Rhabanusstr. 3, Bonifazius-Turm A, 55118 Mainz, Germany

²SDU Metaverse Lab, The Maersk Mc-Kinney Møller Institute, University of Southern Denmark, Campusvej 55, 5230 Odense, Denmark

³Department of Orthopaedics and Traumatology, University Medical Center of the Johannes Gutenberg University Mainz, Langenbeckstraße 1, 55131 Mainz, Germany

Corresponding author: Daan Apeldoorn (e-mail: daan.apeldoorn@uni-mainz.de).

Parts of this paper have been funded through prize money received for the AbstractSwarm project in 2017, provided by the Centrum for Entrepreneurship (CET) of TU Dortmund University and the Industrial Research Foundation. Moreover, special thanks go to the TOPML project, funded by the Carl Zeiss Foundation, grant number P2021-02-014, which helped with finishing the paper during the last phase, especially with Section IV. Furthermore, parts of the work presented in this article are supported by Novo Nordisk Foundation grant NNF25OC0105856.

ABSTRACT This article addresses the optimization of general logistics scenarios that are modeled and simulated in the form of multi-agent systems. The term “general” refers here to the idea of finding highly adaptive algorithms that are not only capable of handling a specific family of scenarios (such as vehicle routing or inventory control problems) but also novel problems that are not known in advance. To foster research in this direction, we ran an international competition on established conferences (IEEE Congress on Evolutionary Computation (CEC) and Genetic and Evolutionary Computation Conference (GECCO)) over four years (with one round per year) based on the AbstractSwarm modeling and simulation system. Participants in the competition were able to submit swarm-based algorithmic implementation (i. e., swarms of individual decision-makers), which we evaluated each year on ten a priori unknown logistics or related scenarios. Novel solutions based on artificial intelligence and swarm-based approaches as well as insights into the difficulties of different kinds of scenarios emerged from the competition. Challenges and results will be presented and applications will be discussed in the context of hospital logistics.

INDEX TERMS artificial intelligence, logistics, multi-agent simulation, optimization, swarm intelligence.

I. INTRODUCTION

An Abstraction of Logistics and Related Problems

MANY logistics and related problems, such as *supply chain management*, *job shop scheduling*, *vehicle routing* scenarios or combinations thereof, are well understood—both from a mathematical and a computer science point of view. Solutions can be provided, e. g., by specific software tools that are geared toward a certain class of problems. However, in highly complex and dynamic environments (e. g., hospital settings), many different kinds of (combined) logistics and related scenarios may be relevant. Moreover, in such settings, even new and a priori unknown scenarios can arise.

The technique of *agent-based* modeling and simulation

(where *agents* are considered entities that can provide their own decisions) allows for providing results for a wide range of logistics scenarios. In addition to that, this simulation technique can also help identify and explain bottlenecks for single entities as well as for the whole process. These features result in two essential requirements:

- An agent-based framework that allows for comprehensible modeling and simulation on an adequate degree of detail.
- Highly adaptive algorithms that are able to find solutions, even in new, a priori unknown scenarios.

While the first requirement is (at least to some extent) already covered by the AbstractSwarm System [1], the second

requirement is still hard to tackle. However, such a degree of adaptivity can be realized by Artificial Intelligence (AI) and Computational Intelligence (CI) methods.

Logistics Scenarios for AI/CI Research and Education The development of approaches that are able to solve a variety of different problems in a more *general* way can be considered one of the long-term research goals of AI/CI methods. Closely following the motivation of the AbstractSwarm competition [2], problem scenarios from the broad field of logistics [3] may serve as eligible test environments for this purpose. Such scenarios can be: extremely diverse, highly dynamic, and varying both in size and complexity. Furthermore, according to the AbstractSwarm Multi-Agent Logistics Competition's aims and scope [2], many logistics-related problems are easy to comprehend but hard to solve [4], which renders them an approachable task for beginners offering diverse learning opportunities.

Competition and Underlying Framework The AbstractSwarm modeling and simulation system (cf. the first proposal by Apeldoorn [1]) serves here as the underlying framework for our competition [5]. The framework provides tools for encoding logistics and related problems and implementing agents that serve as decision-making entities within a simulation. Instead of studying logistics problems from a birds-eye perspective, we are especially interested in fostering the cooperation of independent agents. This also allows for creating simulations on a single-agent level, e. g., for identifying bottlenecks in a scenario. Therefore, we divide the complexity of an overarching planner that controls multiple agents and distribute the complexity among the agents which each need to act based on their own observations and knowledge of the task. Such an agent-centered view puts special demands on the agents' planning, communication, and collaboration skills.

The main goal of the competition is to stimulate AI research in the context of multi-agent and swarm systems as cooperative problem-solvers for a priori unknown problems with a focus on logistics. For the past four years each year, we proposed a set of benchmark scenarios that map prototypical problems (for example see Fig. 1). Solutions are evaluated based on their overall performance on different evaluation problems that cover the diversity and varying complexity of given optimization tasks. Next to that, during the past years, we stimulated both bachelor's and master's theses tackling these modern challenges of agent-centered cooperation.

Unlike other cooperative multi-agent competitions such as the *Multi-Agent Programming Contest* [6] or the *RoboCup Soccer Simulation League* [7], the presented competition focuses on *generality*, i. e., on agent implementations that are not only capable of optimizing one single scenario but *multiple a priori unknown* logistics scenarios. The competition thereby builds on one of the central features of the AbstractSwarm framework: the separation of the scenario description (i. e., the task that has to be done by the agents)

from the implementation of the agent behavior (i. e., how agents can in general solve different kinds of scenarios). In contrast, the well-known agent-related *General Video Game Artificial Intelligence* (GVGAI) competition [8] also focuses on generality but is mainly dedicated to single-agent scenarios.

Contributions This article presents several results in the context of the AbstractSwarm *Multi-Agent Logistics Competition* from the years 2022 to 2024. The main contributions are:

- An analysis of major agent implementations submitted to the competition.
- An investigation of the difficulty of (several properties of) different kinds of logistics scenarios.

Moreover, the performance of a winning agent implementation will be demonstrated in the context of a hospital logistics scenario modeled from a real-world setting.

Outline In Section II, we review the AbstractSwarm framework and describe some of the basics needed for the competition (including the agent programming interface). Section III introduces the AbstractSwarm competition and provides an overview of some example scenarios. This section also comprises the competition's evaluation (Section III-B), as well as the presentation of the competition results (Section III-C). Section IV shows how the framework can be applied to a hospital logistics scenario derived from a real-world setting and provides additional results in this context. Finally, Section V concludes the article.

II. THE ABSTRACTSWARM FRAMEWORK

An initial work within the context of the AbstractSwarm framework was introduced by Apeldoorn [1]. There, a graphical modeling language is proposed for modeling and simulating multi-agent systems for a wide range of problem domains, with a special focus on logistics-related optimization problems. By separating the definition of the agents' task from the definition of the agent behavior (one of the central features of the AbstractSwarm framework), developed agent models can be efficiently reused. In the following subsections, the components of the modeling language are only briefly summarized, closely following Apeldoorn's original proposal [1]. For more detailed considerations, the reader is referred to the original work.

A. GRAPH

In the AbstractSwarm framework, a problem is modeled as a graph:

$$G := (\Phi, E)$$

where Φ is a set of *perspectives* and E is a set of edges. A perspective contains two kinds of *component types*: a set of *agent types* S_{T_A} and *station types* S_{T_S} . Agents represent the problem's mobile entities, while stations represent stationary entities. The set E represents relations among the component

types, which can be categorized in *time edges* E_T , *place edges* E_P , and *visit edges* E_V for modeling temporal, spatial, and visit relations. The respective types of components and relations are introduced in the following.

B. AGENTS, STATIONS, AND ATTRIBUTES

According to Apeldoorn [1], an agent type $T_A \in S_{T_A}$ is defined as a tuple

$$T_A := (t_A, P_A, A)$$

where t_A is a type name, P_A is a set of attributes, and A is a set of agents belonging to said type. In a similar way, a station type $T_S \in S_{T_S}$ is defined as

$$T_S := (t_S, P_S, S)$$

where t_S is a type name, P_S is a set of attributes, and S is a set of stations belonging to said type. In the AbstractSwarm framework, agent types are visualized as circles, including a label in the circle's center and the annotated number of agents in the upper left area. In the same way, station types are visualized as squares.

Agent types and station types can be annotated with *attributes* to refine the modeling. Examples are the *Frequency* or *Necessity* attributes (for limiting the number of visits), the *time* attribute (to limit how long an agent has to visit a station), or the *Space* attribute (to limit the available space at a station); see also Section II-E. A detailed description of the different attributes can be found in the original proposal by Apeldoorn [1]. Examples of different agent types and station types (including annotated attributes) can be found in Figure 1.

C. RELATIONS AMONG AGENT AND STATION TYPES

Next, we take a look at the relations among these entities, still following the conception by Apeldoorn [1]. Edges represent visit, spatial, and temporal relations:

- *Visit Edges*:

A visit edge is an undirected, unweighted edge determining that the respective agents have to visit the connected stations. Visit edges can be marked bold, this forces the agents to start by visiting the station connected with a bold visit edge and can then move to other stations from there. In this sense, an agent's scope of action is determined by the visit edges of its type.

- *Place Edges*:

A place edge is a directed or undirected weighted edge that models the spatial dependencies among stations. This can be used for defining distances and reachability among stations.

- *Time Edges*:

A time edge is a directed or undirected weighted edge that can be used to define temporal relations among the types of agents and stations, e. g., for defining a temporal order of visits or for claiming a delay between visits events of the connected components.

Examples of the different kinds of edges can be found in Fig. 1 (for a more detailed description of each scenario, see Section III-A).

D. PERSPECTIVES

Perspectives can be used to group multiple nodes. Perspectives are a modeling-only element that can help to decompose more complex scenarios (see Fig. 1a and 1g). Since perspectives do not provide any semantics for the simulation, they are not further considered here.

E. SIMULATION

Throughout the simulation, agents visit stations according to their decisions and thereby trigger *visit events*. The visit events define which agent is located at which station at which discrete point in time. In this way, Gantt charts (see, e. g., as used by Pinedo [9]) can be easily retrieved as timetables, by simply recording the visit events in a tabular form: Agents can be enumerated as rows and columns represent the discrete time units. The more compact a timetable is (i. e., the fewer empty cells exist in the timetable), the better the optimization of the scenario is being considered. Fig. 2 shows an example timetable of the scenario from Fig. 1d.

To prevent bias, the calculation order of all agents of a scenario is randomized. Agents can learn about the decisions made by other agents calculated before by making use of the framework's agent communication functionality (which will be explained in more detail in Section II-F2).

The AbstractSwarm software provides scenario modeling, agent implementation, and simulation through a graphical user interface as well as a command line interface for running simulations [10].

F. IMPLEMENTATION OF AGENTS

AbstractSwarm offers a simple and lightweight agent programming interface for the implementation of agents. The current version of the interface is Java-based and comprises only three methods:

- *Evaluation* method (for implementing the action selection of agents)
- *Communication* method (for implementing the communication between agents)
- *Reward* method (for implementing the processing of rewards provided to the agents for their actions)

In the following, the three methods of the agent interface will be considered more detailed (similar definitions can be found in earlier works).

1) Action Selection

The evaluation method allows for the implementation of an agent's action selection, i. e., which station will be chosen by the agent as the next target. The method is called when the agent has no target station, i. e., at the beginning of a simulation run and every time after finishing a visit event (i. e., the visit of a station).

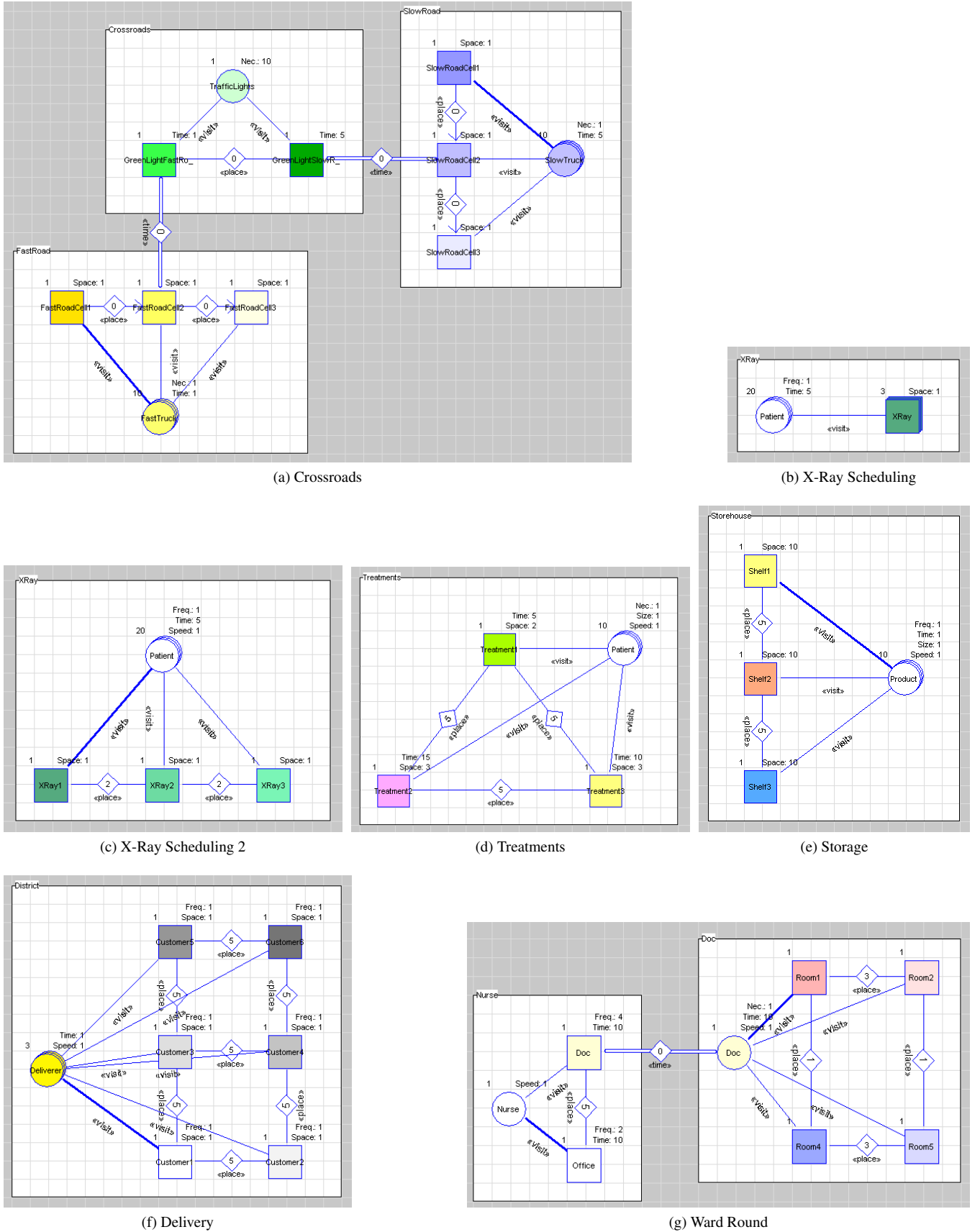


FIGURE 1. Test scenarios from the competition's website [2]

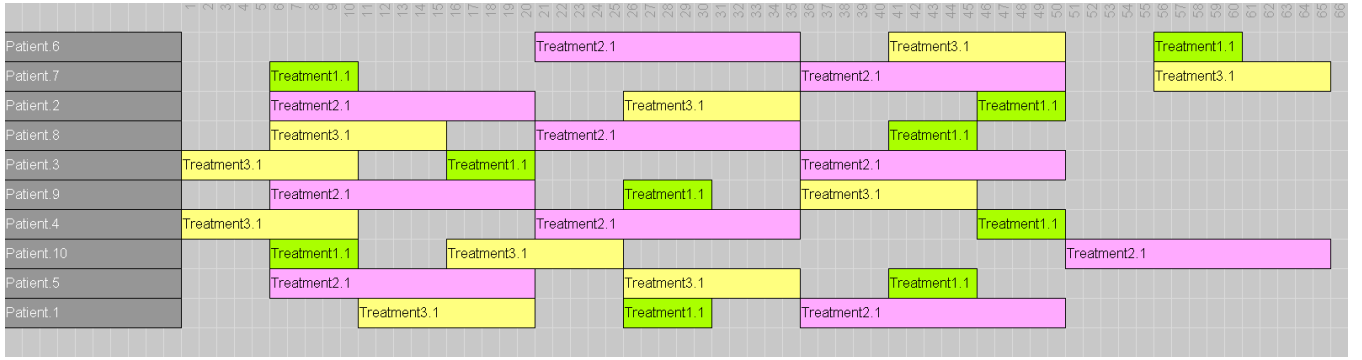


FIGURE 2. Timetable of the *Treatments Scenario* from Fig. 1d (Created with Randomly Deciding Agents)

The method provides the current state of the environment to the agent (including information about all other agents) and it is called for every station that can potentially be selected as the next target by the agent. The method allows for returning a numeric value for the provided station representing the agent's estimate of how good it will be to choose this station as the next target. The station for which the highest value is returned will then be selected as the agent's next target.

The method has the following signature:

$$\text{evaluation}(a, \mathbb{O}_G, \mathbb{S}_G, t, s)$$

where a is the decision-making agent, $\mathbb{O}_G$ is a hash map that associates all other agents $\mathbb{A}_G \setminus \{a\}$ in graph G with an object representing their currently communicated information, $\mathbb{S}$ is the set of all stations in G , t is the current time unit of the simulation and $s \in \mathbb{S}_G$ is the station to be evaluated.

2) Communication

The communication method allows for implementing the communication between agents, i.e., what an agent communicates to other agents. It especially serves for passing information about decisions, plans, etc. to other agents whose evaluation is performed subsequently (e.g., in the same time step or later).

The method is called subsequently to an agent's action selection, i.e., after the agent's evaluation method has been called for each potential target station (see Section II-F1). Similar to the evaluation method, it provides the current state of the environment (including all other agents) to the agent as well as a triple of basic information that is often useful for minimal communication functionality. Any kind of communication information can be returned as an object by the communication method—when only using the minimal communication functionality, this can be the provided triple.

The signature of the method is as follows:

$$\text{communication}(a, \mathbb{O}_G, \mathbb{S}_G, t, d)$$

where a is the communicating agent, $\mathbb{O}_G$ is a hash map that associates all other agents $\mathbb{A}_G \setminus \{a\}$ in graph G with an object representing their currently communicated information, $\mathbb{S}$ is the set of all stations in G , t is the current time unit of the

simulation and d is the default communication data triple $d := (s_d, t_d, e_d)$ with s_d being the station selected by a , t_d being the time when s_d will be reached by a , and e_d being the evaluation value provided by a for s_d .

3) Reward

The reward method allows for processing rewards that are provided by the framework to the agent during the simulation of a scenario. It is called every time an agent finishes an action, i.e., after completing the visit to the last selected station. Since the method is dedicated to the processing of incoming rewards, it has no return value.

The signature of the method is as follows:

$$\text{reward}(a, \mathbb{O}_G, \mathbb{S}_G, t, r)$$

where a is the rewarded agent, $\mathbb{O}_G$ is a hash map that associates all other agents $\mathbb{A}_G \setminus \{a\}$ in graph G with an object representing their currently communicated information, $\mathbb{S}$ is the set of all stations in G , t is the current time unit of the simulation and r is the reward provided to the agent a for its last action, i.e., after the visit of last selected station has been finished. The reward is calculated as the ratio of the number of all agents currently visiting a station divided by the number of all agents in the scenario—averaged over the time of decision-making until the time when the visit of the selected station is completed.

4) Example Agent Implementation

This section provides a small agent implementation to get an intuition of how the agent interface works. The agent implementation uses the *evaluation* and the *communication* method from the AbstractSwarm agent interface only (see Section II-F1 and Section II-F2).

In the implementation provided here, the agent always decides on the station with the maximum free space as its next target. This behavior makes sense, e.g., for scenarios where agents are queuing in front of different stations. For reasons of simplification, the implementation does not consider the time needed to reach a station. The implementation is shown in Algorithm 1.

In the evaluation method, the calculated free space is initialized with the current free space of the station to be

Algorithm 1 Example Implementation of Agents Deciding for the Station with the Maximum Free Space as Next Target Using the AbstractSwarm Java Agent Interface

```

1: public static double evaluation( Agent me,
2:   HashMap<Agent, Object> others,
3:   List<Station> stations, long time,
4:   Station station )
5: {
6:   // Initialize the free space with the station's
7:   // current remaining space
8:   double freeSpace = station.space;
9:
10:  // If the station is not limited in space,
11:  // free space will be infinite
12:  if( station.space == -1 )
13:    return Double.POSITIVE_INFINITY;
14:
15:  // Iterate over all other agents
16:  for( Agent other : others.keySet() )
17:  {
18:    // If another agent has communicated
19:    // the current station as a target
20:    if( (others.get( other ) != null) &&
21:        ((Object[]) others.get( other ))[ 0 ]
22:        == station )
23:    {
24:      // If the agent has limited size,
25:      // subtract from free space
26:      if( other.type.size != -1 )
27:        freeSpace -= other.type.size;
28:
29:      // ...else remaining free space will be
30:      // completely covered
31:      else if( freeSpace > 0 )
32:        freeSpace = 0;
33:    }
34:  }
35:
36:  return freeSpace;
37: }
38:
39: public static Object communication( Agent me,
40:   HashMap<Agent, Object> others,
41:   List<Station> stations, long time,
42:   Object[] defaultData )
43: {
44:   return defaultData;
45: }
46:
47: public static void reward( Agent me,
48:   HashMap<Agent, Object> others,
49:   List<Station> stations, long time,
50:   double value )
51: {
52: }

```

evaluated. If the space of the station is not limited by a *Space* attribute (cf. Section II-B), an infinitely high evaluation value is directly returned for the station.

Otherwise, the remaining free space will be determined by iterating over all other agents and subtracting the size of each agent (that communicated to choose the station) from the station's current free space. In case an agent is not limited in its size by a *Size* attribute, the free space of the station will be completely covered.

In the communication method, only the default communication data (i.e., the selected station, the time when it will be reached, and the agent's evaluation value) is returned. By exploiting this information made available by the other agents in the evaluation method, the free space used for the evaluation of a station also reflects agents that will be at the station in a future state. (For reasons of simplification, the time needed to reach a station is not considered here.)

The implementation of the reward function is left empty due to the simplicity of the agent implementation, no reward for learning, adapting, or improving the agent behavior is processed here.

III. ABSTRACTSWARM COMPETITION

As can be seen in the previous section (Section II-F4), simple agent implementations that are useful for a small subset of possible scenarios can be created easily. However, since the AbstractSwarm framework allows for the modeling of diverse scenarios, we are interested in agents that can be used for the simulation of many different kinds of scenarios—even new ones that are a priori unknown.

The AbstractSwarm Multi-Agent Logistics Competition was initiated in 2021 with the aim of stimulating research for finding such kinds of generic agent implementation. While in the first year, we had no submissions yet and thus used the time to advertise the competition on renowned conferences¹, over the past years, the competition generated 19 official submissions (and one additional late submission not considered for evaluation) by overall 20 participants. Moreover, two bachelor's theses and one master's thesis as well as two resulting papers from participants (Winkel 2022 [11], Schmid & Becker 2023 [12]) emerged in the context of the competition.

For getting started and for training and testing purposes of newly created agent implementations, the competition website provides small episodic example scenarios of different (simplified) logistics (and related) problems. Fig. 1 shows these scenarios (a more detailed description of each scenario will be provided later in Section III-A). The scenario graphs have a similar size compared to the ones that are used later in the evaluation phase of the competition. In addition, further scenario graphs can be created easily using the AbstractSwarm modeling and simulation software [10]. Note, that not all syntax elements are (fully) supported yet by the simulation system and therefore not all syntax elements

¹Genetic and Evolutionary Computation Conference (GECCO) and IEEE World Congress on Computational Intelligence (WCCI)

are relevant to the competition (which slightly simplifies the competition). The following elements were relevant for the three completed competition rounds from 2022–2024 (with possible slight changes over the years) and should have been considered by submitted agents (according to the given competition framework [2]):

- Agent types and station types
- Visit edges, bold visit edges
- Directed and undirected place edges (without *Item* and *Capacity* attributes)
- Time edges (with a weight of zero only)
- Directed Time edges (with weight > 1 only!)
- Attributes: *Frequency*, *Necessity*, *Time*, *Space*, *Size*, *Speed* (Speed may be assumed to be 1 for all agents in scenarios incorporating spacial distance, i.e., scenarios involving place edges with a weight > 0)

A. EXAMPLE BENCHMARK SCENARIOS

As mentioned in the previous section already, small episodic example scenarios of (simplified) logistics (and related) problems are provided on the AbstractSwarm competition website [2] (see Fig. 1). These have been chosen with diversity in mind and address various prototypical scenarios. The scenarios will be shortly summarized in the following by providing the descriptions from the competition website [2]:

- **Crossroads:** Trucks on two different roads (a slow one and a fast one) have to get from a starting point to a destination. The roads cross in the middle, where an agent has to control the traffic lights to optimize the traffic flow (colored in green, see Fig. 1a). *Traffic flow control problem with directed spatial dependencies and a cell-based character (where stations represent the cells of the roads).*
- **X-Ray Scheduling:** Twenty patients that have to be x-rayed must be distributed to three x-ray stations (see Fig. 1b). *Multi-agent scheduling problem.*
- **X-Ray Scheduling 2:** Same as X-Ray Scheduling with distances among the three x-ray stations being additionally modeled (see Fig. 1c). *Multi-agent scheduling problem with spatial dependencies.*
- **Treatments:** Ten patients have to get three different treatments each. The treatment locations have different amounts of space and have a distance of five units from each other (see Fig. 1d). *Spatio-temporal scheduling problem.*
- **Storage:** The best way of storing ten products in three shelves of a linearly organized storehouse must be found here (where the products are represented by agents). The solution is obviously to store all ten products on the first shelf (colored in yellow) since it is located at the entrance of the storehouse and thus distances for storing can be minimized (see Fig. 1e). *Storage problem with sparse rewards (agents only receive better rewards if all agents decide to avoid one of the shelves located in the back).*

- **Delivery:** Three deliverers must coordinate with each other to provide one delivery for each of the six customers in a district. The deliverers start at the bottom left customer (colored in white, see Fig. 1f). *Multi-agent delivery problem.*
- **Ward Round:** A doctor has to do a ward round to visit four patient rooms in a hallway. A nurse has to do some office work but must be available every time the doctor visits a room (see Fig. 1g). *Spatio-temporal coordination problem with only two agents.*

B. EVALUATION

The evaluation of agent implementations in the context of the AbstractSwarm competition is realized according to the *best-found solution* in conjunction with the *time needed* by an agent implementation to find it. In this context, the best solution of a scenario is defined as the solution with the lowest *total waiting time* (i. e., idle time) of all agents in the scenario. Since the competition especially aims at the generality of agent implementations, the evaluation will be averaged over multiple previously unknown scenarios (usually a set of about ten different small-sized scenarios).

Since the evaluation scenarios might be of different sizes regarding the needed total waiting time of all agents, the evaluation must be adequately normalized for each scenario to avoid bias. For this purpose, the total waiting time found by an agent implementation in a simulation run is divided by a roughly estimated maximum total waiting time for the respective scenario. According to [13], the estimated maximum total waiting time $\max_tw\tau(G)$ of a graph G representing the scenario is calculated as follows:

$$\begin{aligned} \max_tw\tau(G) := & \sum_{a \in \mathbb{A}_G} \sum_{s \in \mathbb{S}_G} (|\mathbb{A}_G| - \text{sim_visits}(s)) \cdot \frac{\text{visits}(a, s)}{\text{sim_visits}(s)} \cdot \text{time}(a, s) \\ & + \sum_{a \in \mathbb{A}_G} \sum_{s \in \mathbb{S}_G} \text{visits}(a, s) \cdot \max_travel(a, s) \\ & + \sum_{a \in \mathbb{A}_G} \sum_{s \in \mathbb{S}_G} \text{visits}(a, s) \cdot \max_wait(a, s) \end{aligned} \quad (1)$$

where

- $\mathbb{A}$ is the set of all agents and in graph G
- $\mathbb{S}$ is the set of all stations in graph G
- $\text{visits}(a, s)$ is the number of visits an agent a has to perform on a station s in the scenario represented by the graph G
- $\text{sim_visits}(s)$ is the number of simultaneous visits that can be performed on a station s
- $\text{time}(a, s)$ is the time needed by an agent a at station s (i. e., the number of time units of the visit event)
- $\max_travel(a, s)$ is the maximum number of time units needed by an agent a to reach station s (the summed weights of all place edges are considered here as an approximation), and
- $\max_wait(a, s)$ is the maximum number of time units an agent has to wait according to time edges before station s

can be entered (i. e., the highest weight of all time edges connected to station s).

The intuition here is that the maximum total waiting time can be estimated by considering the maximum time possibly needed by each agent to reach and visit its stations, assuming that all other agents have to wait during that time.

The provided formula should be considered a rough estimate only, which aims at providing a basic normalization of agent results to be able to balance the comparison of agents in the context of scenarios of different sizes. Note that $\text{visits}(a, s)$, $\text{sim_visits}(s)$ as well as $\text{time}(a, s)$ may also require further estimations: E. g., in case of $\text{visits}(a, s)$, if the type of agent a defines a frequency attribute while being connected to several stations, it is not clear how often the station s will be visited or whether it will be visited at all. In such cases, the frequency attribute can be divided by the number of connected stations to get the estimate of $\text{visits}(a, s)$ (which might lead to values < 1).

Example This example considers the scenario *Treatments*, one of the test scenarios from the competition's website [2] (see Section III-A and Fig. 1d). In this scenario, ten patients have to get three different treatments each. The treatments have a spatial distance of five units from each other (cf. Section III-A). Applying Formula (1), results in the following normalization value for this scenario:

$$\begin{aligned}
 \text{max_tw}_t(G_{\text{Treatments}}) &= \\
 &10 \cdot \underbrace{\left((10-2) \cdot \frac{1}{2} \cdot 5 \cdot 1 + (10-3) \cdot \frac{1}{3} \cdot 10 \cdot 1 + (10-3) \cdot \frac{1}{3} \cdot 15 \cdot 1 \right)}_{\text{treatment 1 station} \quad \text{treatment 2 station} \quad \text{treatment 3 station}} \\
 &+ 10 \cdot 3 \cdot \underbrace{1 \cdot 15}_{\text{place edges}} \\
 &+ 10 \cdot 3 \cdot \underbrace{0}_{\text{time edges}} \\
 &= 1233.\bar{3}
 \end{aligned}$$

The first line after the equality sign represents the sum of the potential worst-case waiting times of all ten agents when performing each of the three treatments while all other agents are assumed to wait. The second line concerns the waiting time resulting in the worst-case from place edges for every single visit event. The third and last line before the result concerns the waiting time resulting from time edges with non-zero weights (which is zero here, since there are no such time edges in this scenario).

Table 1 shows the worst case total waiting times estimates calculated for each of the test scenarios described in Section III-A according to Formula (1).

C. RESULTS

At first, as a proof-of-concept for the evaluation approach described in Section III-B, the subsequent section (Section III-C1) will provide a benchmark of some baseline agent implementations by evaluating them in the context of the scenarios described in Section III-A. After that, the main

TABLE 1. Worst Case Total Waiting Time Estimates provided by Formula (1) for the Scenarios from Section III-A

Scenario	Worst Case Total Waiting Time Estimate (rounded to integer)
Crossroads	4800
X-Ray Scheduling	1900
X-Ray Scheduling 2	1980
Treatments	1233
Storage	100
Delivery	222
Ward Round	162

results of the competition are presented for each round from 2022–2024 (Section III-C2). In addition, an analysis of the scenarios' difficulty will be provided for each year (Section III-C3). Finally, the results will be discussed in Section III-C4.

1) Baseline Agents

This section provides some baseline results of basic agent implementations delivered with the AbstractSwarm framework: Randomly deciding agents ("Random"), agents searching for the shortest paths to their next target station ("MinDistance"), as well as agents considering the maximum available free space for their decisions ("MaxFreeSpace"). In the following, the agent implementations will be briefly described. After that, the evaluation of the agents in the context of the scenarios from Section III-A will be provided.

- *Random:*

The Random agent makes random decisions. If it has to decide on the next target station, it assigns a random value to each potential station.

- *MinDistance:*

The MinDistance agent evaluates the distance between itself and the potential next target stations. The shorter the distance to a station, the better it is rated by the agent as next target.

- *MaxFreeSpace:*

The MaxFreeSpace agent evaluates stations according to the available free space. The more free space a station offers, the better the agent rates it as next target (see Algorithm 1 for the implementation).

To evaluate the described agent implementation in the context of the scenarios described in Section III-A, the agent implementations were each run for 100 runs over 10 repetitions in each scenario from Section III-A. The results are normalized using Formula (1) and the area under the resulting curve is considered, and averaged over all scenarios. The area is used here as an evaluation criterion since both the agents' best-found solution as well as the time needed to find should be taken into account for evaluation.

In case an agent implementation is not able to solve a certain scenario (as it is the case for *Crossroads*, which cannot be solved by any of the three considered agent implementations due to the directed place edges being involved), the maximum total waiting time calculated according to Formula (1) is considered, resulting in a normalized value of 100 (i. e., 1 multiplied by 100 runs).

Fig. 3 shows the results, where agents selecting their next target station according to the minimal distance perform best among the three considered baseline implementations.

2) Competition Results 2022–2024

All agent implementations submitted to one of the AbstractSwarm competition rounds were evaluated on a set of ten scenarios. These scenarios were chosen with diversity in mind to cover different kinds of logistics aspects. Six of the scenarios were modified versions of already existing scenarios (half of them being scaled and half of them being structurally modified) and four scenarios were newly created for a competition round. However, for each round, all ten scenarios were published only after the submission phase and thereby were unknown to the participants in advance. Each agent implementation was evaluated for 100 runs on each of the ten scenarios using the evaluation scheme outlined in Section III-B. The results were averaged over 20 repetitions for each agent implementation running on a scenario and over all ten scenarios for each agent implementation.

Over the three years, 19 agents have been submitted to the competition, and discussing all of them would exceed the scope of this paper. However, in the following, an overview of different problem-solving strategies used by some of the agent submissions will be briefly summarized. The submitted implementations can be mainly categorized into two groups. (For more detailed information, the agent implementations can be downloaded from the AbstractSwarm competition website [2].)

- *Hard-coded agents:*

This group of submissions simply follows a static implementation for making decisions. No learning from previous runs is involved (i. e., the framework's reward method is usually not used or at least the provided reward is ignored). The group comprises the baseline agents that are shipped with the AbstractSwarm framework, described in Section III-C1, as well as some of the submissions. Such kinds of agents might or might not implement an agent-based communication (cf. Algorithm 1).

- *Learning agents:*

In this group of submissions, agents are able to learn from previous runs to improve their performance over multiple runs. To achieve this, the framework's reward method can be used. Different learning techniques have been used by participants, such as statistical techniques (Schmid & Becker 2023 [12]) as well as reinforcement learning and self-organizing maps (Winkel 2022 [11]).

Apart from these two groups, in the last two years, some *meta-learning* agents joined the competition. These agents learn over (some of) the other agent implementations (also other learning agents) to figure out, which agent implementation fits best to which scenario.

Fig. 4 shows the results of the competition from 2022–2024. The *LearningAgent* agent implementation (Schmid & Becker 2022 [12]) shows a constantly good performance over the years and won the competition in 2022 and 2023. Only in 2024, it was slightly outperformed by *ABMetaLearnerMod* submitted by Halstenberg. Note that, since the competition results are based on diverse a priori unknown scenarios, agent implementations performing better in the competition are supposed to be more generic and to perform better in practice as well.

All agent implementations of each competition round are published under the GNU Public License (GPL) version 3 and can be downloaded from the competition website [2].

3) Difficulty Analysis of Evaluation Scenarios

For the assessment of the AbstractSwarm competition, each agent's performance is measured for the previously unknown 10 evaluation scenarios of the respective year. Regrouping that data by evaluation scenario gives an insight into the difficulty of the different scenarios, as can be seen in Fig. 5. Each evaluation scenario was given a name based on its content, prefixed with either "NEW", "CHANGED", or "SCALED" to indicate the scenario's origin. A NEW scenario was created from scratch. CHANGED and SCALED scenarios are based on existing scenarios (either a benchmark scenario or an evaluation scenario from a previous year) that were then either scaled (e. g., more agents added) or changed (e. g., weighted place edges added).

The created scenarios for evaluating the competition offer a diverse set of challenges each year. The distribution of the scenario difficulties within one year was rather similar over the different competition years, with 2 to 3 very difficult scenarios (total area > 65) but also 2 to 3 rather simple scenarios (total area < 20).

Looking at the average area of each year's evaluation scenario set shows that the 2022 competition round (average area ≈ 37.47) and the 2023 competition round (average area ≈ 37.03) offer a very similar degree of difficulty. For the 2024 competition round, we tried to slightly increase the difficulty of the scenarios, which is reflected in an average area of ≈ 40.46 . However, note that this indication is of limited validity as the average performance of submitted agents might also have changed over the years.

The evaluation scenario sets of each competition round can be downloaded from the competition website [2].

4) Discussion of Results

This section provides a discussion of all the results presented in the previous sections (Section III-C1 to Section III-C3).

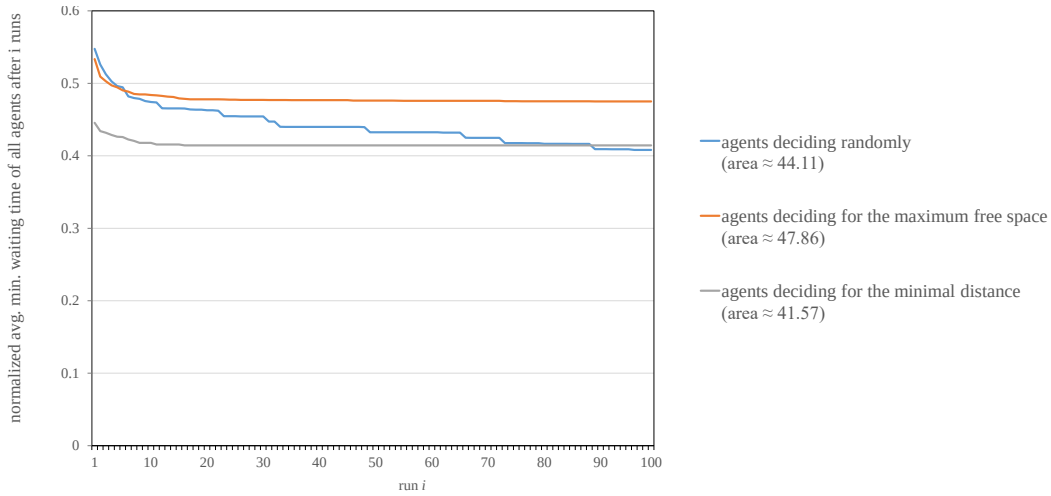


FIGURE 3. Performance Curves of the Agent Implementations from Section III-C1 Averaged Over All Scenarios from Section III-A with 100 Runs and 10 Repetitions each

Baseline Agents Considering the simple example agent implementations from Section III-C1 with a basic decision-making mechanisms, it can be observed in Fig. 3 that the three implementations provide rather similar results. However, the agents that always consider the station with the minimal distance as the next target provide the best performance overall scenarios from Section III-A. This seems to be the case due to the *Storage* scenario for which the *MinDistance* agent implementation finds the optimal solution immediately and therefore the normalized area of the plot equals 0. This makes sense, since the best solution is to place all the products (here represented by the agents) on the shelf closest to the bold visit edge, which is exactly what the agent *MinDistance* does by selecting the station with minimal distance. In contrast, this agent implementation performs poor for the *XRayScheduling2* scenario, where the agents end up queuing in front of the *XRay1* station.

Interestingly, the randomly deciding agents provide the second-best solution here: Random decisions may help to explore many different solutions independently from the scenario. However, this has obvious limitations, e.g., when it comes to scenarios where specific constraints have to be fulfilled, as induced, e.g., by directed place edges (as in case of the *Crossroads* scenario).

Finally, the agents basing their decisions on the maximum free space on the potential next target stations provide the worst overall results. Although this approach seems to be intuitive for preventing queuing, it is rather specific and does not consider further constraints (such as spatial or temporal dependencies).

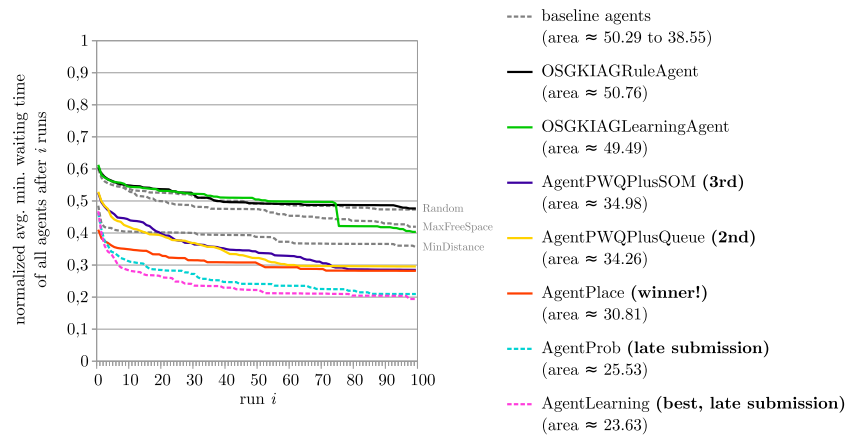
In the 2022 round of the competition, all three of these rather simple agents were evaluated alongside the submitted agents (Fig. 4 to provide a baseline compared to the competition entries. The randomly deciding *AgentRandom* implementation was additionally evaluated for all

three years. Comparing its results to the submissions, almost all agent submissions perform better. Only one agent (*OSG_KIAG_RuleAgent*) in 2022 was slightly outperformed by *AgentRandom* (by an area difference of ≈ 0.47).

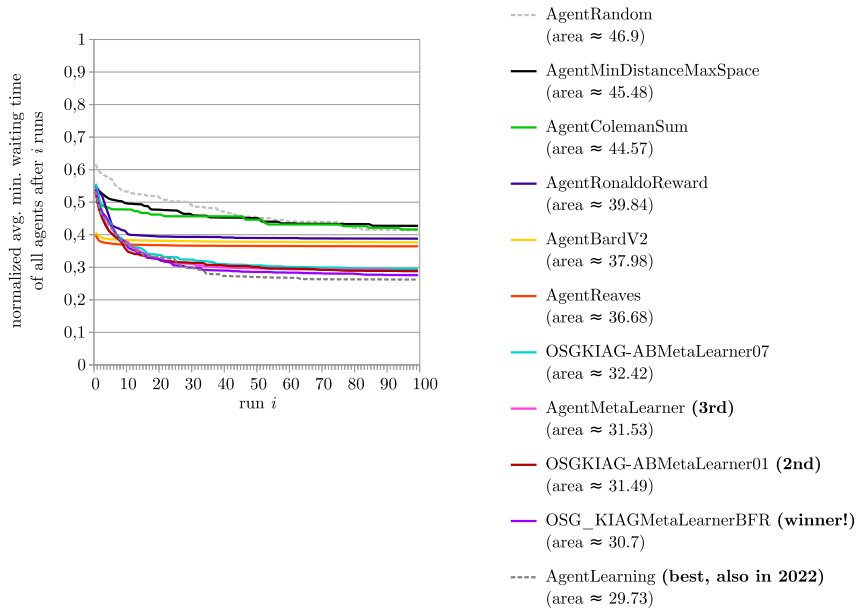
Competition Results In the 2022 round of the competition (see Fig. 4a), while most submitted agents were better than the baseline implementations, the three implementations submitted by Schmid (*AgentPlace*, *AgentProb* and *AgentLearning*) clearly outperformed all other agents. (However, being late submissions, *AgentProb* and *AgentLearning* were not included in the official ranking.)

In 2023, basically three groups of agent implementations can be identified in the plot of Fig. III-C2, with the strongest group mainly consisting of meta-learning approaches (i.e., agent implementations that learn over (some of the) implementations of the previous round(s) to find the best approach for a scenario). However, again, *AgentLearning* is the best agent implementation (which was only included as a benchmark here and was therefore not considered in the official ranking of this round).

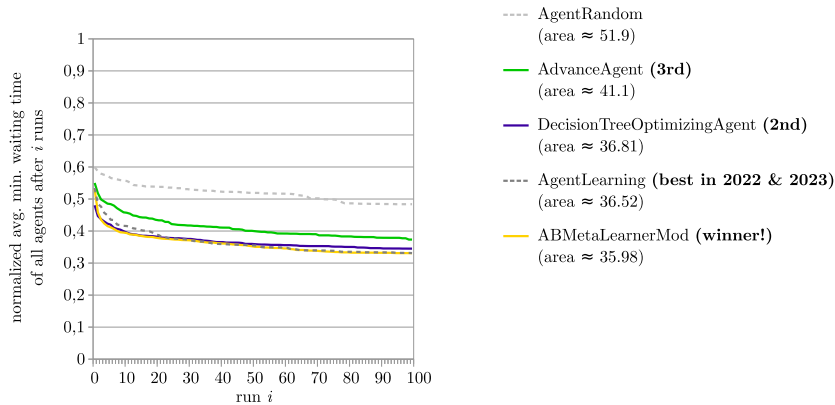
In the most recent round of the competition in 2024 (see Fig. III-C2), the meta-learner implementation *ABMetaLearnerMod* submitted by Halstenberg was able to outperform the long-term best implementation *AgentLearning*. At a first glance, this might sound intuitive, since learning over previously submitted agent implementations also includes the top-performing agents from the previous round(s). However, although being the best agent implementation in 2024, *ABMetaLearnerMod* only slightly outperformed the best (non-meta) learning agent implementation *AgentLearning* of this year (i.e., the area difference between both agent implementations was only ≈ 0.54). Since meta-learning agents have to explore different agent implementations in each scenario, they can spend less runs for the training of



(a) Agent Performance Evaluation 2022



(b) Agent Performance Evaluation 2023



(c) Agent Performance Evaluation 2024

FIGURE 4. Results of AbstractSwarm Multi-Agent Logistics Competition Submissions 2022–2024

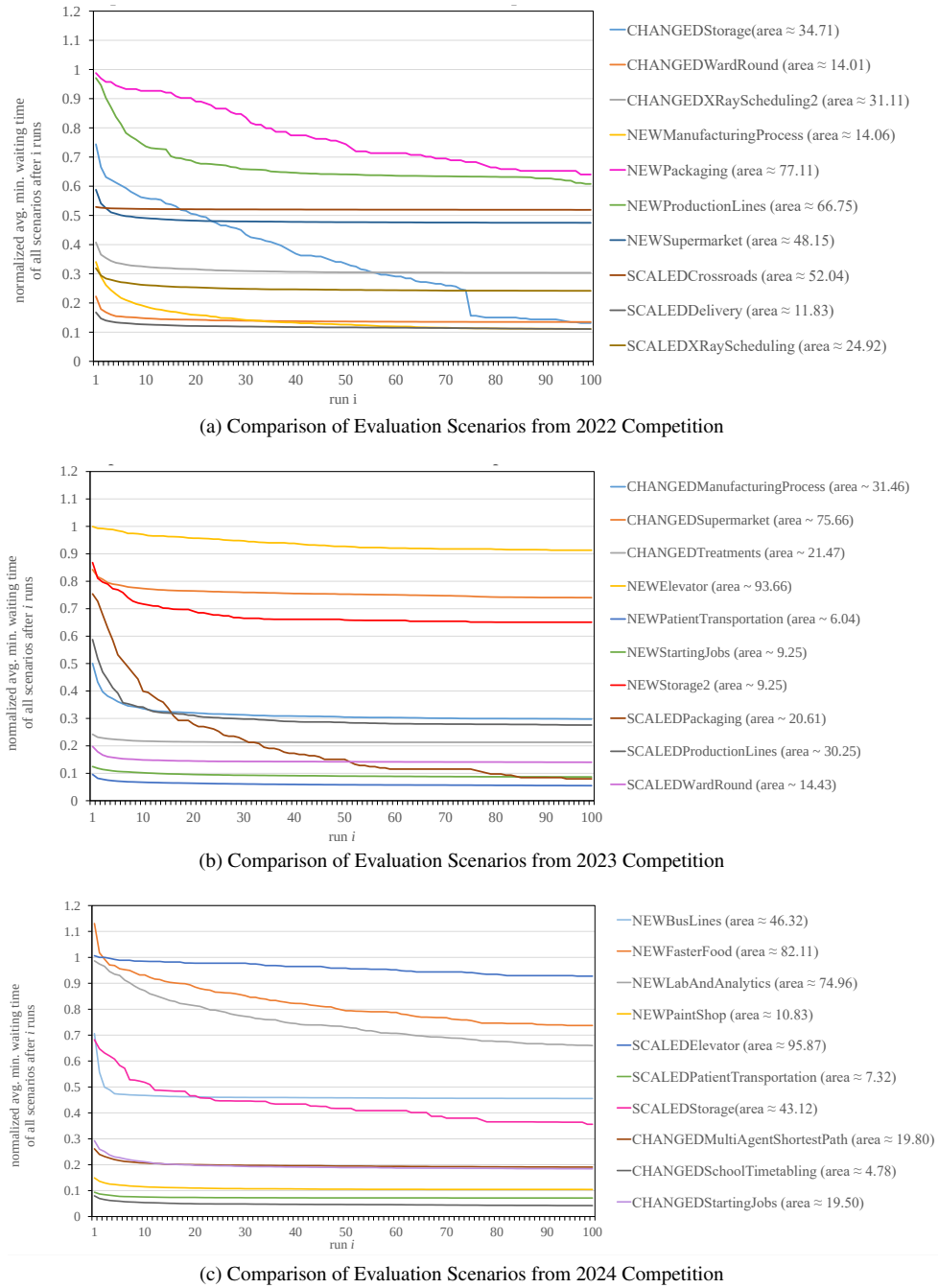


FIGURE 5. Difficulty Analysis of the Evaluation Scenarios from the AbstractSwarm Competition

each implementation per scenario. This could explain why meta-learning agents are not necessarily better (as can also be seen for all meta-learners from the competition round of 2023).

Difficulty of Evaluation Scenarios Finally, the results concerning the difficulty of the different competition scenarios will be discussed here. (Note that the comparability of the scenario difficulties over the different competition rounds is rather limited, since the set of agent implementations changed each year, with only a few agent implementations being in-

cluded again.)

According to the difficulty analysis from Section III-C3 and Fig. 5, the most difficult scenario in 2023 was *Elevator* and in the following round of 2024 a scaled version of the scenario (*SCALED Elevator*) scenario was also identified as the hardest scenario to solve for the submitted agents. A reason for this might be the complexity of the scenario, since it combines many (directed) spatial and temporal dependencies. Thus, agents have to perform well on both these tasks individually and in combination in order to find a good solution.

Apart from that, it can also be the case that a low number

of place and time edges do not result in an easy-to-handle scenario: As an example, the scenario *NEWPackaging* from the 2022 round of the competition was the most difficult of this year. It features no temporal and only one undirected spatial dependency. The difficulty of this scenario is that two stacks of products have to be distributed into two packages—there is an optimal solution that fits all products in their respective package. However, if only one product ends up in a wrong place (i. e., if one agent is making a wrong decision), this solution cannot be found anymore.

IV. APPLICATION IN A HOSPITAL LOGISTICS SCENARIO

As an example, this section applies parts of the presented results in a model derived from a real-world hospital logistics use case from the University Medical Center of the Johannes Gutenberg University Mainz in Germany. For this purpose, the most promising agent implementation submitted to the AbstractSwarm competition will be used for simulations. By this means, the applicability of the approach for increasing the *transparency* of such a scenario (i. e., providing insights into its structure, potential problems and how they can be solved) will be demonstrated. Within the simulations, no personal data is required, which minimizes the potential risk of any *privacy* issues. First, the scenario description will be provided (Section IV-A). After that, the key question will be described (Section IV-B). Finally, the simulation model will be introduced (Section IV-C) and the experiments and their results will be presented (Section IV-D).

A. SCENARIO DESCRIPTION

In the considered scenario, the focus is on a clinical center of the University Medical Center of the Johannes Gutenberg University Mainz. The center experiences problems with patient transportation, especially in the case of transportation from a patient's room to a surgery room. This might cause extensive waiting times for both patients and professionals. The patient transportation is realized by a service shared with further clinical institutions. Every patient transportation request allows for providing a priority from: $\{low, mid, high\}$. Since it is well-known that patient transports might take an extensive amount of time, it seems that most of the patient transportation requests are intentionally done with an increased priority level (both from the clinical center that is in the focus here as well as from the other clinical institutions using the patient transportation service). This leads to the assumption that the requests with increased priority level might be part of the problem itself. (Note that the goal of modeling and simulating the described scenario is not primarily the optimization of the patient transportation service but the investigation of *reasons* and *explanations* for being able to tackle the experienced problems efficiently.)

B. KEY QUESTIONS FOR FINDING REASONS AND EXPLANATIONS

To investigate reasons and explanations for the patient transportation problems, the described scenario will be modeled

as AbstractSwarm graph with the following key questions in mind:

- How does a proper choice of priorities affect the patient transportation times?
- Given a proper choice of priorities, are there better alternative strategies for the processing of the prioritized patient transportation requests?

While the first question concerns the behavior of both members of the focused clinical center as well as those of the other clinics using the patient transportation service, the second question concerns the patient transportation service itself.

C. SIMULATION MODEL

For the creation of the simulation model, assumptions have been derived from acquired data as well as from an exchange with the clinical center that is in the focus of this use case (including an inspection of the clinical center). The following assumptions found their way into the model. The parameters directly used in the simulation model marked in **bold** (note that the focus is here is on the transports of patients to and from a surgery room):

- 14 patient transports per day to and from a surgery room (based on the median of data from an exemplary week of the clinical center in focus) are transported to their target and back to their respective patient room, resulting in $2 \cdot 14 = \mathbf{28}$ patient transports per day.
- From the 28 patients transports per day to and from a surgery room, $20\% \approx 6$ are assumed to have a need for an increased priority and $10\% \approx 3$ are assumed to have a need for a high-priority transport. However, since it can be assumed that the way back never has a need for priority transportation, $6 : 2 = \mathbf{3}$ mid-priority and $3 : 2 \approx \mathbf{2}$ (rounded up) high-priority transports are needed, resulting in $28 - (3 + 2) = \mathbf{23}$ remaining low-priority transports.
- 10 other clinics using the patient transportation service (based on exchange with a medical doctor partly involved in the scenario).
- Clinics are assumed to have an equal need for patient transports, resulting in a daily need for $23 \cdot 10 = \mathbf{230}$ low-priority, $3 \cdot 10 = \mathbf{30}$ mid-priority, and $2 \cdot 10 = \mathbf{20}$ high-priority patient transports.
- The individual duration of patient transports t_{trans} is not considered here. Thus, t_{trans} will be assumed to be 1.

Fig. 6 provides an overview of the complete model of the patient transportation scenario. Although Figure 6 does not allow for inspecting each perspective's inner details, it provides an overview of the model. According to Fig. 6, the simulation model is separated into eight perspectives:

- The white perspective and the red perspective in the center of Fig. 6, model the incoming requests from the other clinics (white) and the clinical center in the focus (red). The darker the red, the higher the priority provided by an incoming request.

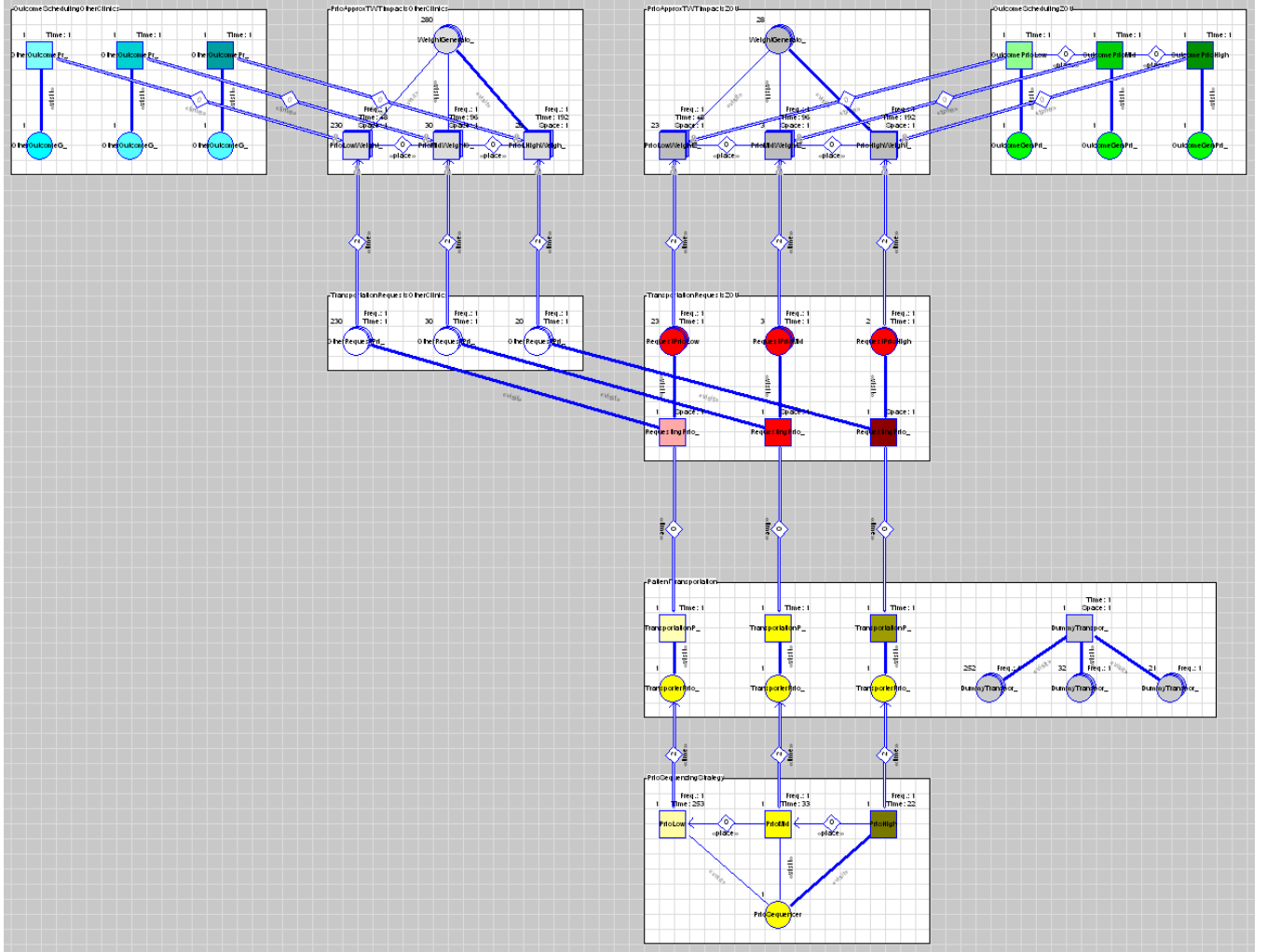


FIGURE 6. Overview of the AbstractSwarm simulation model for the patient transportation scenario. Red/white: incoming patient transportation requests by the focused clinical center (red) and other clinics (white), green/cyan: outcome patient transports, yellow: patient transportation service processing strategy and realization of transports, gray: additional technical aspects. Note that the graphic only serves to provide an overview of the model and therefore abstracts from details such as names and attribute values of entities (more detailed explanations are provided in Section IV-C).

- The two perspectives in the top row colored in the cyan and green model the outcome of patient transports for the other clinics (cyan) and the clinical center in the focus (green). The darker the cyan/green, the higher the *actual* priority of a patient's transport.
- The two perspectives colored in yellow represent the patient transportation service: The lower perspective implements the processing strategy used by the patient transportation service for processing the patient transportation requests (e.g., from high to low). The upper perspective implements the actual patient transports (yellow) as well as a small dummy model (gray) needed for technical reasons only to preserve comparability among the different experimental settings of the model.
- The two perspectives in the top row colored in gray are needed for technical reasons to simulate the impact of late patient transports in relation to their respective priorities: By this means, late patient transports with

higher priority will cause a higher impact than late patient transports with lower priority.

D. EXPERIMENTS AND RESULTS

1) Experimental Setup

For answering the key questions from Section IV-B in the context of the simulation model, the best agent implementations from the competition will be used and the overall *total waiting time* of all agents will be considered for evaluation. For this purpose, the impact of late scheduled patient transports will be related to the length of a complete simulation run, in dependence on the patient transport's priority (i.e., patient transports with a higher priority will have a higher impact on the length of a complete simulation run which results in a longer total waiting for all agents up to the end of the run).

Since each patient transportation is considered to take constant time, the length L of a complete simulation run can be roughly estimated by summing up the time needed for the

number of low, mid, and high-priority requests, i. e.,

$$L := t_{trans} \cdot (n_{low} + n_{mid} + n_{high}) \quad (2)$$

with t_{trans} being the patient transportation time and $n_{low}, n_{mid}, n_{high}$ being the number of patient transport requests of the respective priority. (Note that the real length of a simulation run incorporates minor technical delays of a couple of time units which will be omitted here for reasons of simplification.)

To relate a simulation run's length to the impacts of late patient transports, an adequate amount of time has to be assigned to each priority level (with the largest amount of time being assigned to the patient transports with the highest priority). The main idea here is that scheduling a longer event later in a simulation run results in a higher risk of increasing the length of the run than for shorter events. This is because longer events that are scheduled earlier allow for shorter events to be scheduled in parallel while scheduling the longer events later reduces this advantage (since there might be not enough shorter events left to be scheduled in parallel). Note that in the simulation model, this is implemented by the two gray-colored perspectives in the topmost row of Fig. 6, as described in Section IV-C, see last bullet point.

To determine the basic event length l assigned to a low-priority patient transport in this context, the following equation will be considered

$$L \cdot s = w_{high}l + w_{mid}l + l \quad (3)$$

where L refers to the length of a complete simulation run (as defined in Equation 2), $s > 1$ is a scaling factor ensuring that late patient transports have the potential to exceed the length of a simulation run, and w_{high}, w_{mid} are the coefficients determining the relative impact among high-priority, mid-priority and low-priority transports.

For the upcoming experiments, the scaling factor will be set to $s := 1.1$. Late mid-priority patient transports will be considered to have a double impact compared to low-priority patient transports and late high-priority patient transports will be considered to have a double impact compared to late low-priority patient transports, resulting in $w_{mid} := 2$ and $w_{high} := 4$. Solving Equation 3 and inserting the numbers from Section IV-C results in

$$\begin{aligned} l &= \frac{L \cdot s}{w_{high} + w_{mid} + 1} \\ &= \frac{(230 + 23 + 30 + 3 + 20 + 2) \cdot 1.1}{4 + 2 + 1} \approx 48 \end{aligned}$$

time units for the basic event length assigned to a low-priority patient transport (and thus $2 \cdot 48 = 96$ for a mid and $2 \cdot 96 = 192$ for a high-priority patient transport).

Having now related the impact of late patient transports to the overall length of a simulation run, the total waiting time of all agents can be measured to evaluate the upcoming experiments. More concretely, the experiments will be evaluated according to the following function measuring the

overall impact of late patient transports in dependence on their priority:

$$\begin{aligned} \text{tardiness_impact}(G_{PatientTransportation}) &:= \\ \min_tw\tau(G_{PatientTransportation}) & \\ - \min_tw\tau(G'_{PatientTransportation}) & \quad (4) \end{aligned}$$

where $G_{PatientTransportation}$ is the graph of the simulation model for the patient transportation scenario (see Fig. 6), $G'_{PatientTransportation}$ is nearly the same graph without the perspectives related to the impact of late patient transports and the connected outcome perspectives (i. e., without the cyan, green and gray perspectives in the topmost row of Fig. 6), and $\min_tw\tau$ is the minimal total waiting time of all agents found by running the simulation for $r = 100$ runs.

Thus, tardiness_impact refers to the tardiness based on the “overhang” of events resulting from the assigned impact of late patient transports over the basic time needed for a simulation run without these events.

2) Experiments

For answering the key questions from Section IV-B, two kinds of experiments will be conducted (where the first experiment refers to the question of how the use of priorities for patient transportation requests affects the efficiency of the overall process and the second experiment refers to the question whether there are better alternative strategies for the patient transportation service of handling the priorities of the incoming requests).

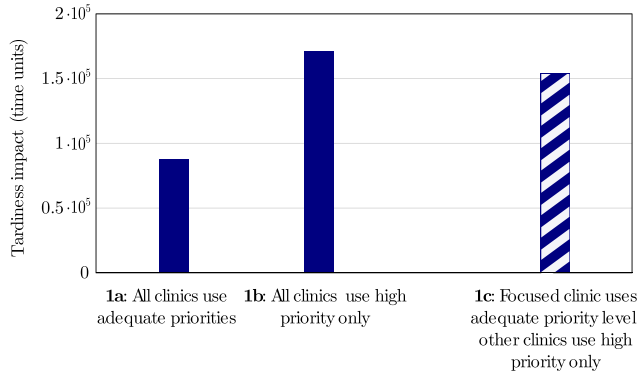
Experiment 1 (Clinics) – How does the use of priorities affect the overall process?

- (1a) The scenario from Fig. 6 will be simulated.
- (1b) The same scenario will be simulated in a modified version with all patient transportation of all clinics being requested with the highest priority (without considering their real priority).
- (1c) Only the focused clinic uses adequate priorities for patient transportation requests according to their real priorities (while all other clinics remain with the highest priority).

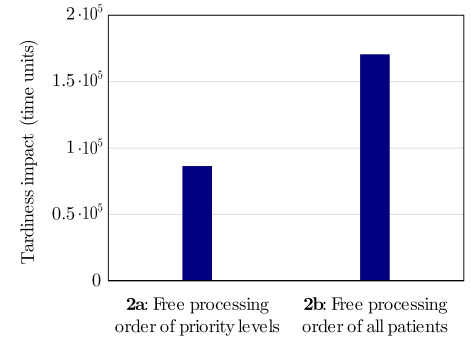
Experiment 2 (Patient transportation service) – Are there better strategies for handling priorities?

- (2a) The scenario from Fig. 6 will be run with the modification that the processing order of the different priorities in the bottommost perspective can be determined freely (instead of having a fixed order).
- (2b) The scenario is modified, such that the processing order of *every single* patient transport can be determined freely.

For both experiments, the agent implementation *AgentLearning* from the competition will be used, which was a successful implementation over multiple years. (Note that the winning agent implementation *ABMetaLearnerMod* from the 2024 roundup of the competition was not able to find a solution in every run, resulting in some technical issues. Therefore,



(a) Results of **Experiment 1a**, **1b** and **1c**: Comparing the results of **1a** and **1b**, using priorities adequately clearly results in the lowest tardiness impact for all clinics—however, even *one* clinic can affect the tardiness impact when sticking to adequate priorities, as shown by the separated hatched bar for **1c**



(b) Results of **Experiment 2a** and **2b**: Trying to optimize the processing order of patient transport requests does not improve the tardiness impact (in case of **2b**, the result was even much worse and agents were not able to find a better processing order than the naive order from high to low within 100 runs)

FIGURE 7. Results from the Hospital Logistics Application Scenario (Figure 6): Each experiment comprises 100 runs with the AgentLearning implementation from the competition

it was not considered here.) All results will be based on the `tardiness_impact` measure from Formula 4.

3) Results

The results of the experiments are shown in Fig. 7.

The results of the first experiment clearly show that both for the focused clinic as well as for all other clinics, it is most efficient to request the patient transportation service with their actual priorities (**1a**). If all clinics are requesting every patient transport on the highest priority level, this renders the patient transportation nearly two times more inefficient. Interestingly, if only the focused clinic uses the actual priorities, while all other clinics request every patient transport on the highest priority (**1c**, see hatched bar in Fig. 7a), the overall process is $> 10\%$ less inefficient compared to the scenario of **1b**. This means that, according to these results (and based on the assumptions from Section IV-C), it seems to be worth providing the actual priorities when requesting patient transports, even if it is assumed that the other clinics are always requesting on the highest priority level, resulting in a more efficient process for *all* clinics.

Concerning the second experiment, it can be seen, that changing the processing order of priorities (**2a**) does not have a large effect on the efficiency of the overall process. In the experiment, the best order found by the agents after 100 runs was *mid* $\rightarrow$ *high* $\rightarrow$ *low*. This might be explained by the fact that, compared to the low priority requests, only few mid/high priority requests are needed. Therefore, there seems to be a small difference in the scheduling order of mid/high priority requests as long as the large amount of low priority requests are scheduled last. Letting the agents freely choose the processing priorities (**2b**) leads to a much less efficient process after 100 runs. A reason for that might be that 100 runs are not enough, given the size of the scenario. This is underpinned by the found order, which, after 100 runs, still

appeared rather randomly. However, this provides some more evidence that the patient transportation service operates rather well (either with the processing order of *high* $\rightarrow$ *mid* $\rightarrow$ *low* or with the order *mid* $\rightarrow$ *high* $\rightarrow$ *low*) and that the larger part of the solution in rendering the overall process more efficient lies in providing the actual priorities to the patient transportation service.

V. CONCLUSION AND FUTURE WORK

This paper presented the results of the AbstractSwarm Multi-Agent Logistics Competition and its application in hospital logistics. The competition, held over four years (three of which providing results), aimed to explore and evaluate the generalizability of agent-based solutions to a wide range of logistics and related problems. Supported by AbstractSwarm's graphical user interface, over the past years, the competition served as a teaching instrument which emerged several bachelor theses and a master thesis from different institutions. In this context, several agent implementations have been contributed by competition participants, providing solutions with different machine learning techniques and further approaches.

Apart from educational purposes, the resulting agents contributed by competition participants allow for simulating and improving different kinds of scenarios. Moreover, the diversity of competition scenarios created over the past years served for identifying challenging (aspects of the) scenarios. By this means, the competition and its results may be relevant for logistics-related simulation scenarios with a need for simulation on a single-agent level (e.g., for special analysis or comprehensibility purposes).

To demonstrate this, successful agents from the competition have been applied to analyze a hospital logistics scenario concerning patient transports for several clinics at the University Medical Center of the Johannes Gutenberg University

Mainz. The simulation-based study revealed key findings regarding the impact of patient transport prioritization on system efficiency. The results demonstrated that (according to the assumptions made) adhering to actual priority levels significantly reduces overall waiting times, while indiscriminately increasing priority requests exacerbates inefficiencies. However, the results also imply that a single clinic can already noticeably contribute to improve the overall performance of the service for all clinics. Considering the latter as motivating result, this might help improve the patient transportation performance.

Overall, our findings underscore the potential of multi-agent simulations in tackling complex logistics challenges. The AbstractSwarm framework provides a flexible and effective environment for testing and refining AI-driven logistics solutions. Future extensions may concern the enhancement of the framework's expressivity. Furthermore, computational aspects should be considered for improving the framework regarding its capabilities of simulating large-scale scenarios. Besides the further development of the framework, future work could also comprise the development of new agent implementations adaptive to potential new extensions.

ACKNOWLEDGMENT

Thanks go to all participants of the AbstractSwarm Multi-Agent Logistics Competition as well as to additional contributors over the past years.

REFERENCES

- [1] D. Apeldoorn, "AbstractSwarm – a generic graphical modeling language for multi-agent systems," in *Multiagent System Technologies*, ser. LNCS, M. Klusch, M. Thimm, and M. Paprzycki, Eds., vol. 8076. Berlin, Heidelberg: Springer, 2013, pp. 180–192. [Online]. Available: https://doi.org/10.1007/978-3-642-40776-5_17
- [2] "AbstractSwarm Multi-Agent Logistics Competition website," Access May 07, 2026. [Online]. Available: https://abstractswarm.gitlab.io/abstractswarm_competition/
- [3] R. N. Boute and M. Udenio, "AI in logistics and supply chain management," *SSRN Electronic Journal*, 2021. [Online]. Available: <https://doi.org/10.2139/ssrn.3862541>
- [4] P. Toth and D. Vigo, *The vehicle routing problem*. Philadelphia: Society for Industrial and Applied Mathematics, 2002.
- [5] D. Apeldoorn, A. Dockhorn, and T. Panholzer, "Abstractswarm multi-agent logistics competition: Multi-agent collaboration for improving a priori unknown logistics scenarios," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, ser. GECCO '24 Companion. New York, NY, USA: Association for Computing Machinery, 2024, p. 1–2. [Online]. Available: <https://doi.org/10.1145/3638530.3664053>
- [6] "Multi-Agent Programming Contest website," Access May 07, 2026. [Online]. Available: <https://multiagentcontest.org/>
- [7] "RoboCup Soccer Simulation League website," Access May 07, 2026. [Online]. Available: <https://ssim.robocup.org/>
- [8] D. Perez-Liebana, S. Samothrakakis, J. Togelius, T. Schaul, and S. Lucas, "General Video Game AI: Competition, Challenges and Opportunities," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.
- [9] M. Pinedo, *Scheduling – Theory, Algorithms, and Systems*. New York, Dodrecht, Heidelberg, London: Springer, 2012.
- [10] "AbstractSwarm simulation system repository," Access May 07, 2026. [Online]. Available: <https://gitlab.com/abstractswarm/abstractswarm>
- [11] P. A. Winkel, "Abstractswarm multi-agent logistics competition entry: Qplus," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, ser. GECCO '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 3–4. [Online]. Available: <https://doi.org/10.1145/3520304.3534054>
- [12] E. Schmid and M. Becker, "Training agents for unknown logistics problems," in *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*, ser. GECCO '23 Companion. New York, NY, USA: Association for Computing Machinery, 2023, p. 243–246. [Online]. Available: <https://doi.org/10.1145/3583133.3590724>
- [13] A. Jenn and F. Linkerhäger, "Evaluation tool," in *AbstractSwarm Multi-Agent Modeling and Simulation – 20 Years of Logistics with Agents and Stations*, D. Apeldoorn, A. Dockhorn, and T. Panholzer, Eds. Springer, to be published in 2026.



DAAN APELDOORN works for the Institute of Medical Biostatistics, Epidemiology and Informatics (IMBEI) at the University Medical Center of the Johannes Gutenberg University Mainz and is a professor of computer science in the dual study program at the IU International University of Applied Sciences, Germany. His research interests focus on agent-based simulations and knowledge representation in the context of learning agents and decision support systems.



ALINA JENN received her B.Sc. in Molecular Biotechnology from Technical University Munich in 2021. Afterwards she proceeded to do an Internship at a pharmaceutical company. Currently she is working as a student assistant at the Medical Informatics department of the Institute of Medical Biostatistics, Epidemiology and Informatics (IMBEI) of the Johannes Gutenberg University Mainz and is completing a M.Sc. in Applied Bioinformatics at Johannes Gutenberg University Mainz.



ALEXANDER DOCKHORN is an Associate Professor at the Metaverse Lab of the University of Southern Denmark (SDU) in Odense, where he works on AI in games and its transfer to broader applications. He earned his B.Sc., M.Sc., and PhD in Computer Science from Otto von Guericke University Magdeburg (OVGU). His postdoctoral work spans Queen Mary University of London and OVGU, followed by a Junior Professorship at Leibniz University Hannover.



JOCHEN WOLLSTÄDTER is a medical doctor working for the Department of Orthopaedics and Traumatology at University Medical Center of the Johannes Gutenberg University Mainz. He has additional qualifications in specific traumatology and specific orthopedic surgery.



FABIAN LINKERHÄGNER worked as a Student Assistant in Medical Informatics at the Institute of Medical Biostatistics, Epidemiology and Informatics (IMBEI) of the Johannes Gutenberg University Mainz while pursuing his Master's degree at the Technical University of Braunschweig. After graduating, he entered the private sector and now works as an engineer in the metal and electronics industry.



TORSTEN PANHOLZER is Head of the Division of Medical Informatics at the Institute of Medical Biostatistics, Epidemiology and Informatics (IMBEI), a department of the University Medical Center at Johannes Gutenberg University Mainz. He holds a PhD in natural sciences from the Johannes Gutenberg University Mainz. Subsequently, he worked as a Data Processing Analyst and Software Engineer at two Tech firms before joining IMBEI as a Research Associate. His current work focuses

on system and data integration, identity management, and artificial intelligence.